

CLOUD COMPUTING LAB

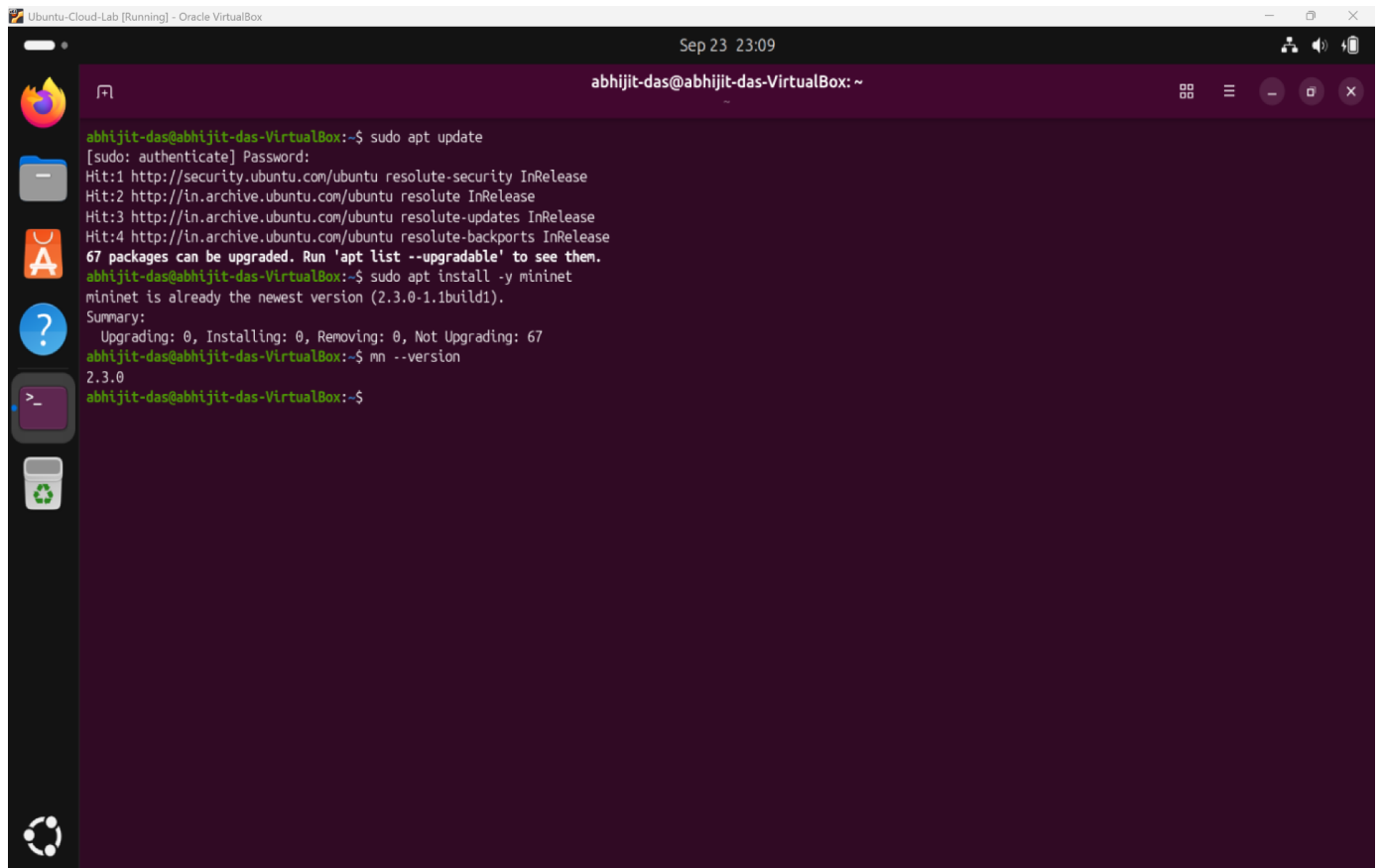
Assignment 3

Creation of Simple Network Topology and Visualization

Mininet Network Topologies

Submitted by: Abhijit Das

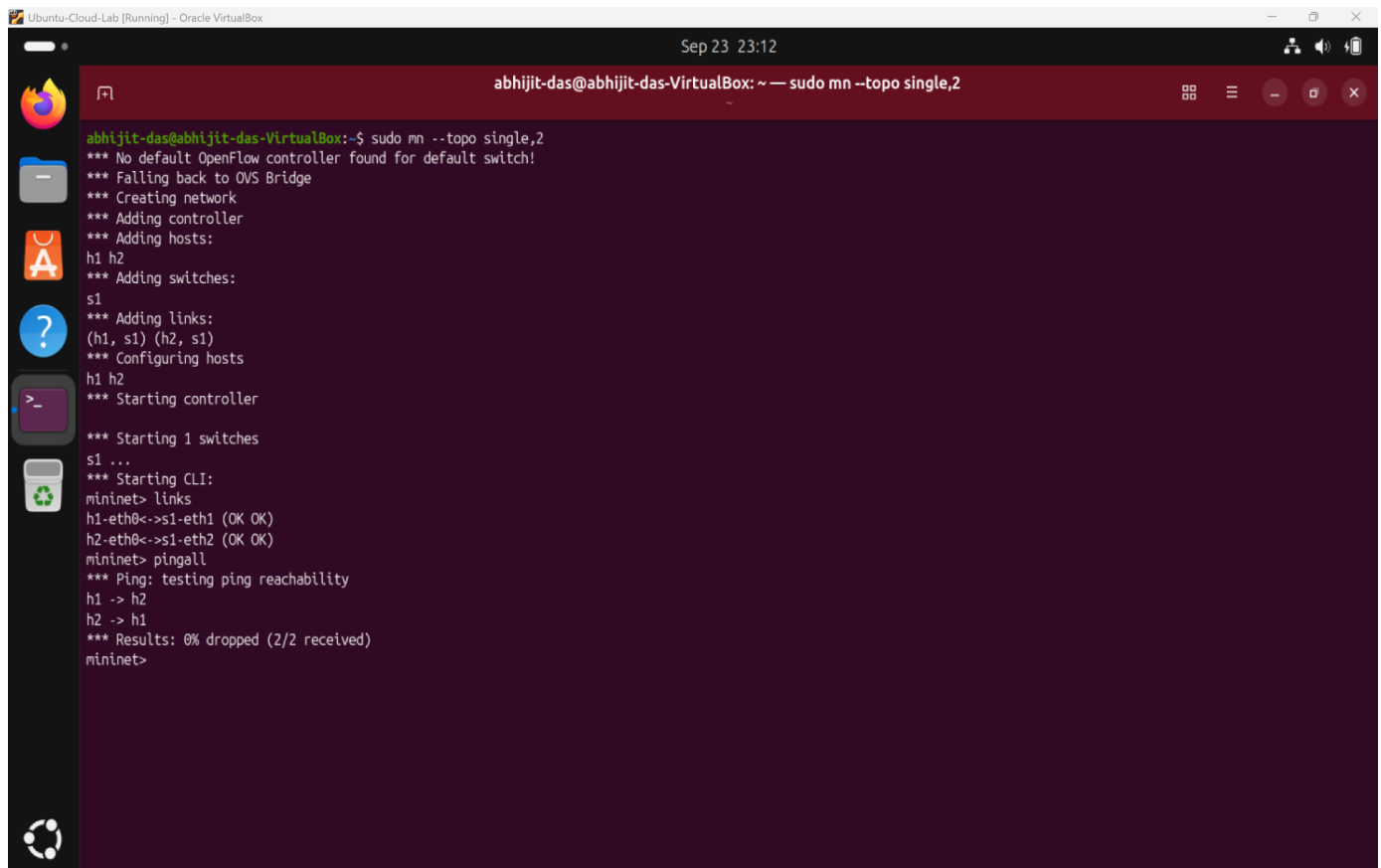
Figure 1: Installation and verification of Mininet in Ubuntu Linux



The screenshot shows a terminal window titled 'Ubuntu-Cloud-Lab [Running] - Oracle VirtualBox' with a timestamp of 'Sep 23 23:09'. The user 'abhijit-das' is logged in at 'abhijit-das@abhijit-das-VirtualBox: ~'. The terminal output shows the following commands and results:

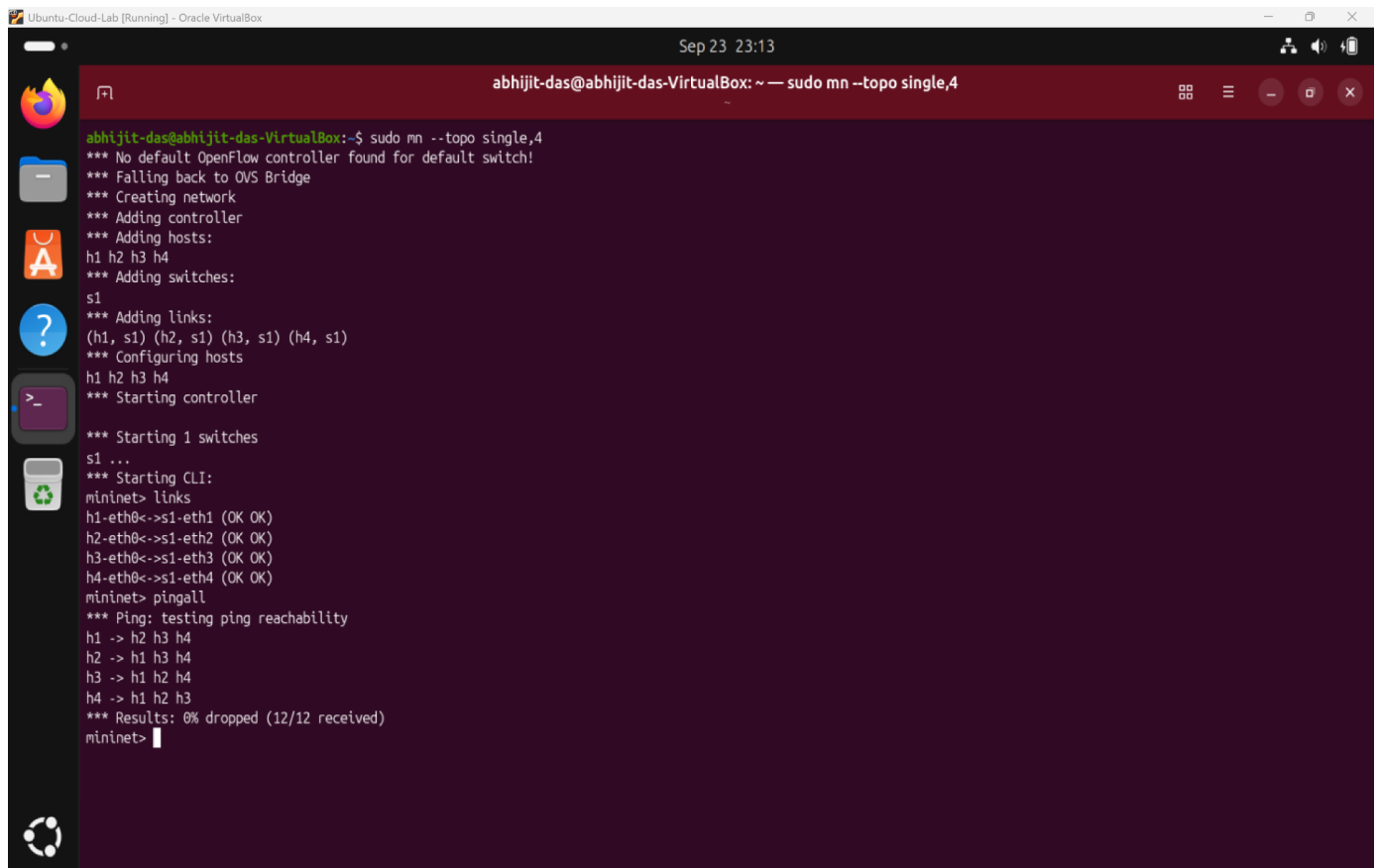
```
abhijit-das@abhijit-das-VirtualBox:~$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu resolute InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu resolute-backports InRelease
67 packages can be upgraded. Run 'apt list --upgradable' to see them.
abhijit-das@abhijit-das-VirtualBox:~$ sudo apt install -y mininet
mininet is already the newest version (2.3.0-1.1build1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 67
abhijit-das@abhijit-das-VirtualBox:~$ mn --version
2.3.0
abhijit-das@abhijit-das-VirtualBox:~$
```

Figure 2: Configuration, links and connectivity verification of Star topology with 2 hosts



```
abhihit-das@abhihit-das-VirtualBox: ~ -- sudo mn --topo single,2
abhihit-das@abhihit-das-VirtualBox:~$ sudo mn --topo single,2
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s1-eth2 (OK OK)
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet>
```

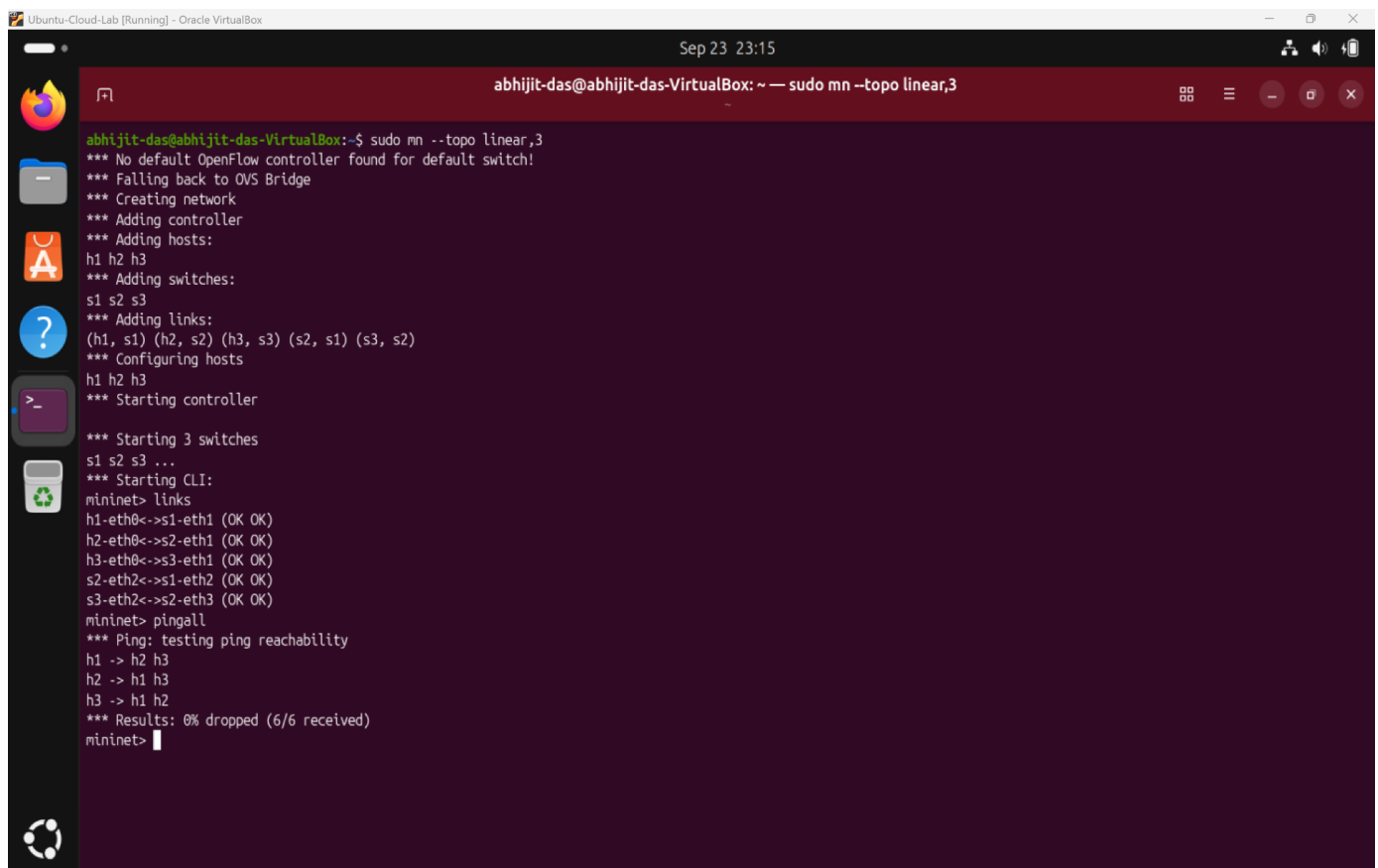
Figure 3: Configuration, links and connectivity verification of Star topology with 4 hosts



The screenshot shows a terminal window titled "abhihit-das@abhihit-das-VirtualBox: ~ — sudo mn --topo single,4". The terminal output displays the following commands and their results:

```
abhihit-das@abhihit-das-VirtualBox:~$ sudo mn --topo single,4
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s1-eth2 (OK OK)
h3-eth0<->s1-eth3 (OK OK)
h4-eth0<->s1-eth4 (OK OK)
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet>
```

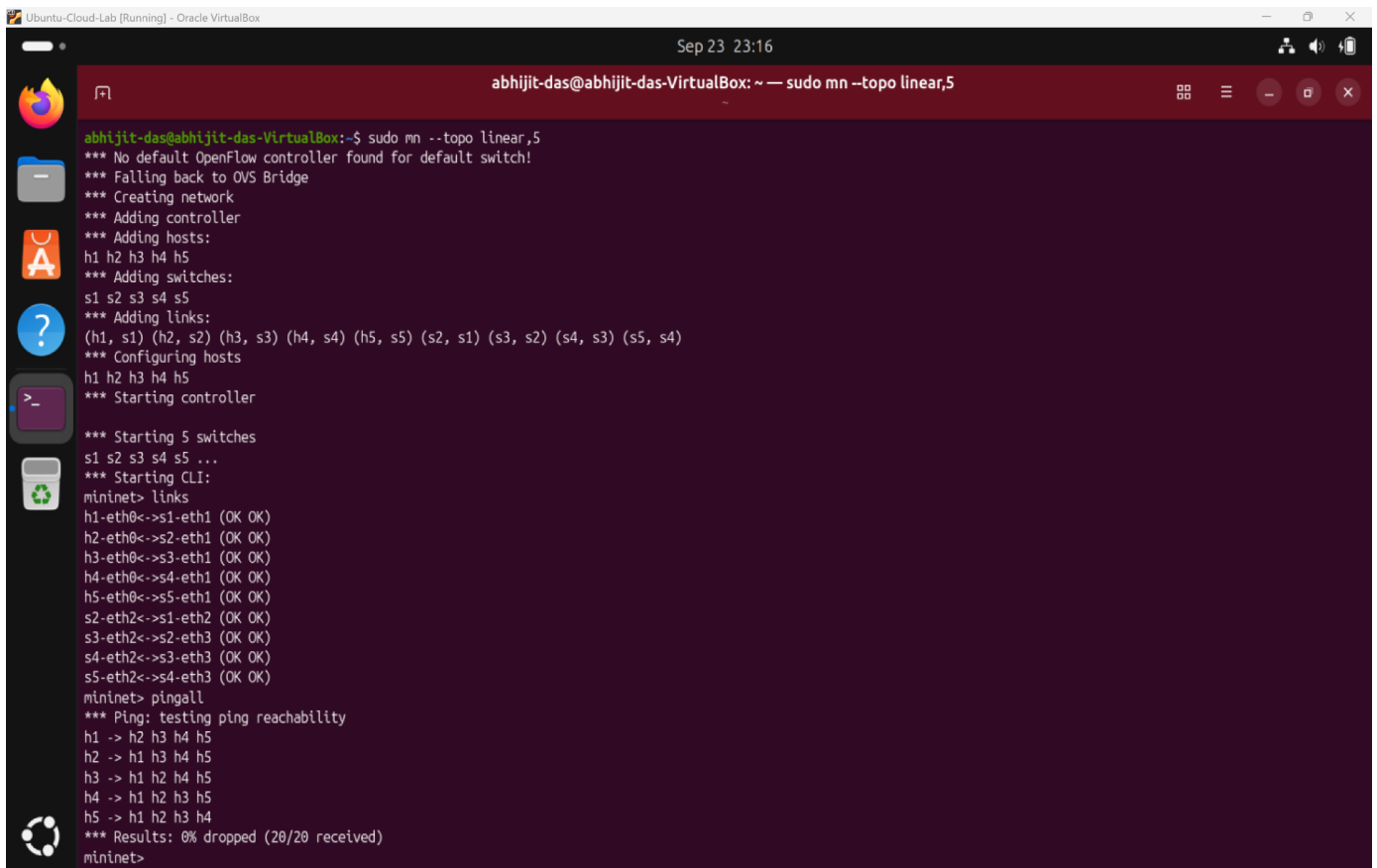
Figure 4: Configuration, links and connectivity verification of Linear topology with 3 hosts



The screenshot shows a terminal window titled "Ubuntu-Cloud-Lab [Running] - Oracle VM VirtualBox" with a timestamp of "Sep 23 23:15". The user is logged in as "abhijit-das@abhijit-das-VirtualBox" and is running the command "sudo mn --topo linear,3". The terminal output shows the following steps:

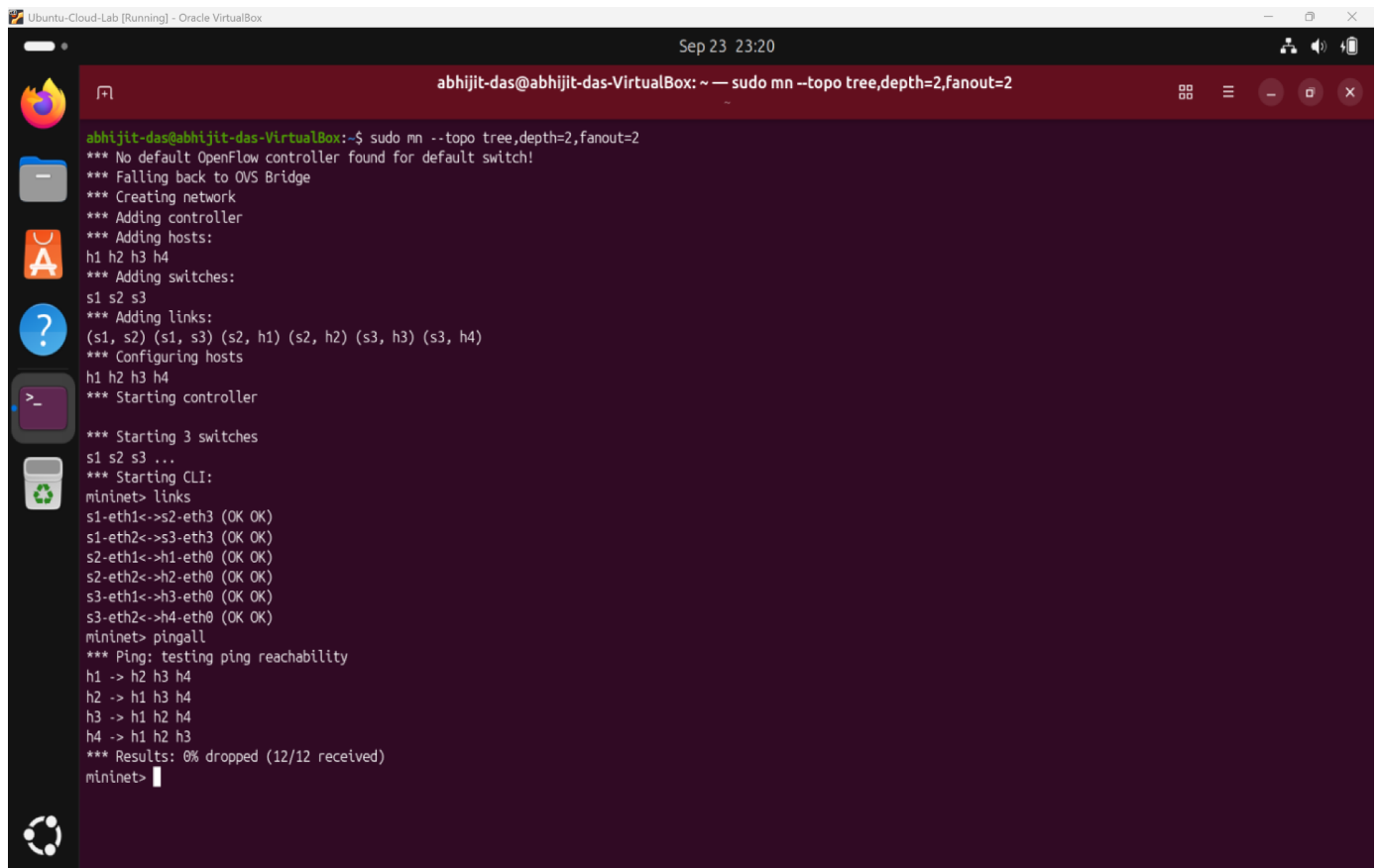
```
abhijit-das@abhijit-das-VirtualBox:~$ sudo mn --topo linear,3
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1 s2 s3
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (s2, s1) (s3, s2)
*** Configuring hosts
h1 h2 h3
*** Starting controller
*** Starting 3 switches
s1 s2 s3 ...
*** Starting CLI:
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s2-eth1 (OK OK)
h3-eth0<->s3-eth1 (OK OK)
s2-eth2<->s1-eth2 (OK OK)
s3-eth2<->s2-eth3 (OK OK)
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet>
```

Figure 5: Configuration, links and connectivity verification of Linear topology with 5 hosts



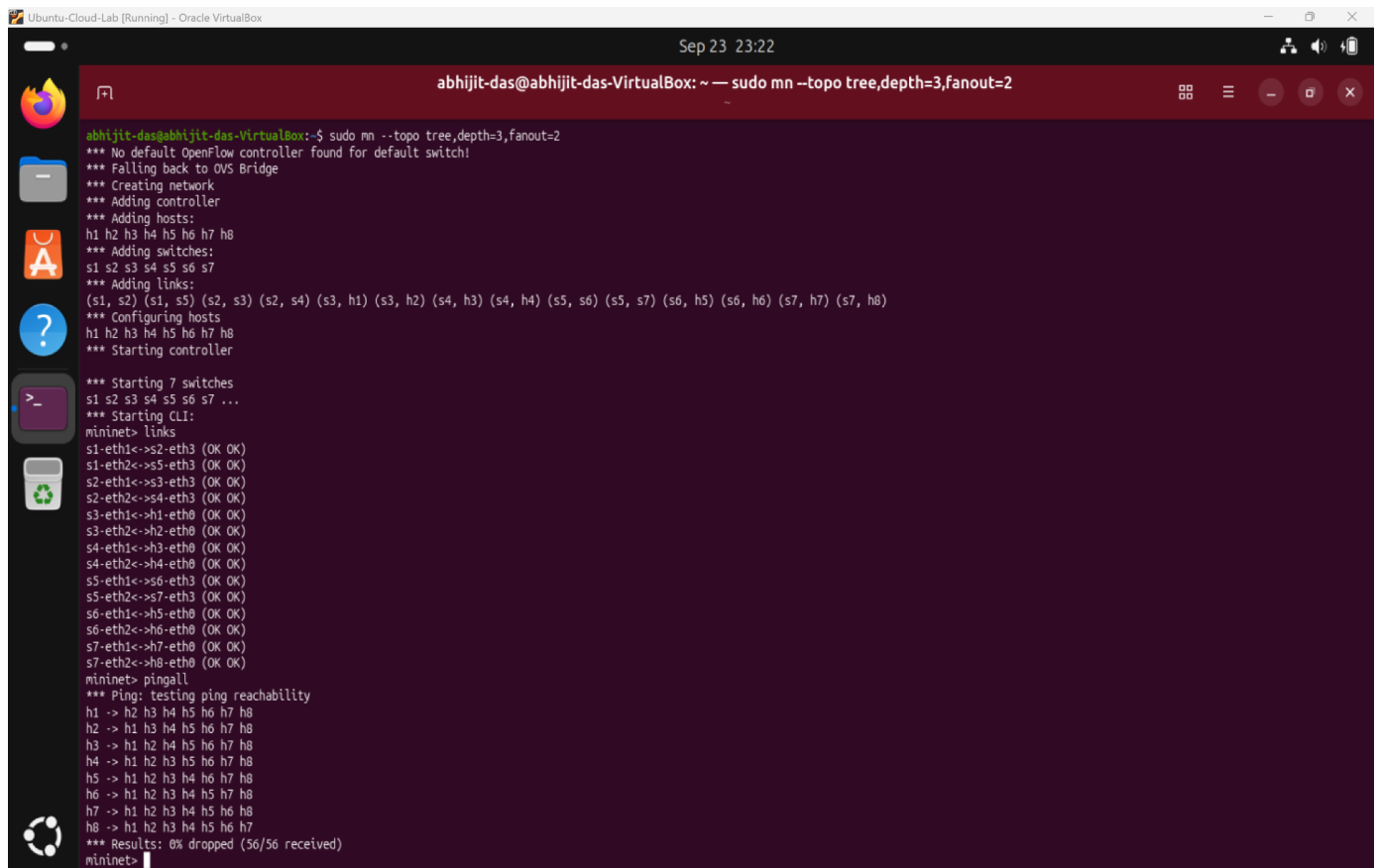
```
abhihit-das@abhihit-das-VirtualBox: ~ -- sudo mn --topo linear,5
abhihit-das@abhihit-das-VirtualBox:~$ sudo mn --topo linear,5
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5
*** Adding switches:
s1 s2 s3 s4 s5
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (h5, s5) (s2, s1) (s3, s2) (s4, s3) (s5, s4)
*** Configuring hosts
h1 h2 h3 h4 h5
*** Starting controller
*** Starting 5 switches
s1 s2 s3 s4 s5 ...
*** Starting CLI:
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s2-eth1 (OK OK)
h3-eth0<->s3-eth1 (OK OK)
h4-eth0<->s4-eth1 (OK OK)
h5-eth0<->s5-eth1 (OK OK)
s2-eth2<->s1-eth2 (OK OK)
s3-eth2<->s2-eth3 (OK OK)
s4-eth2<->s3-eth3 (OK OK)
s5-eth2<->s4-eth3 (OK OK)
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5
h2 -> h1 h3 h4 h5
h3 -> h1 h2 h4 h5
h4 -> h1 h2 h3 h5
h5 -> h1 h2 h3 h4
*** Results: 0% dropped (20/20 received)
mininet>
```

Figure 6: Configuration, links and connectivity verification of Tree topology with 4 hosts



```
abhiжит-das@abhiжит-das-VirtualBox: ~ — sudo mn -topo tree,depth=2,fanout=2
abhiжит-das@abhiжит-das-VirtualBox:~$ sudo mn --topo tree,depth=2,fanout=2
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3
*** Adding links:
(s1, s2) (s1, s3) (s2, h1) (s2, h2) (s3, h3) (s3, h4)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
*** Starting 3 switches
s1 s2 s3 ...
*** Starting CLI:
mininet> links
s1-eth1<->s2-eth3 (OK OK)
s1-eth2<->s3-eth3 (OK OK)
s2-eth1<->h1-eth0 (OK OK)
s2-eth2<->h2-eth0 (OK OK)
s3-eth1<->h3-eth0 (OK OK)
s3-eth2<->h4-eth0 (OK OK)
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet>
```

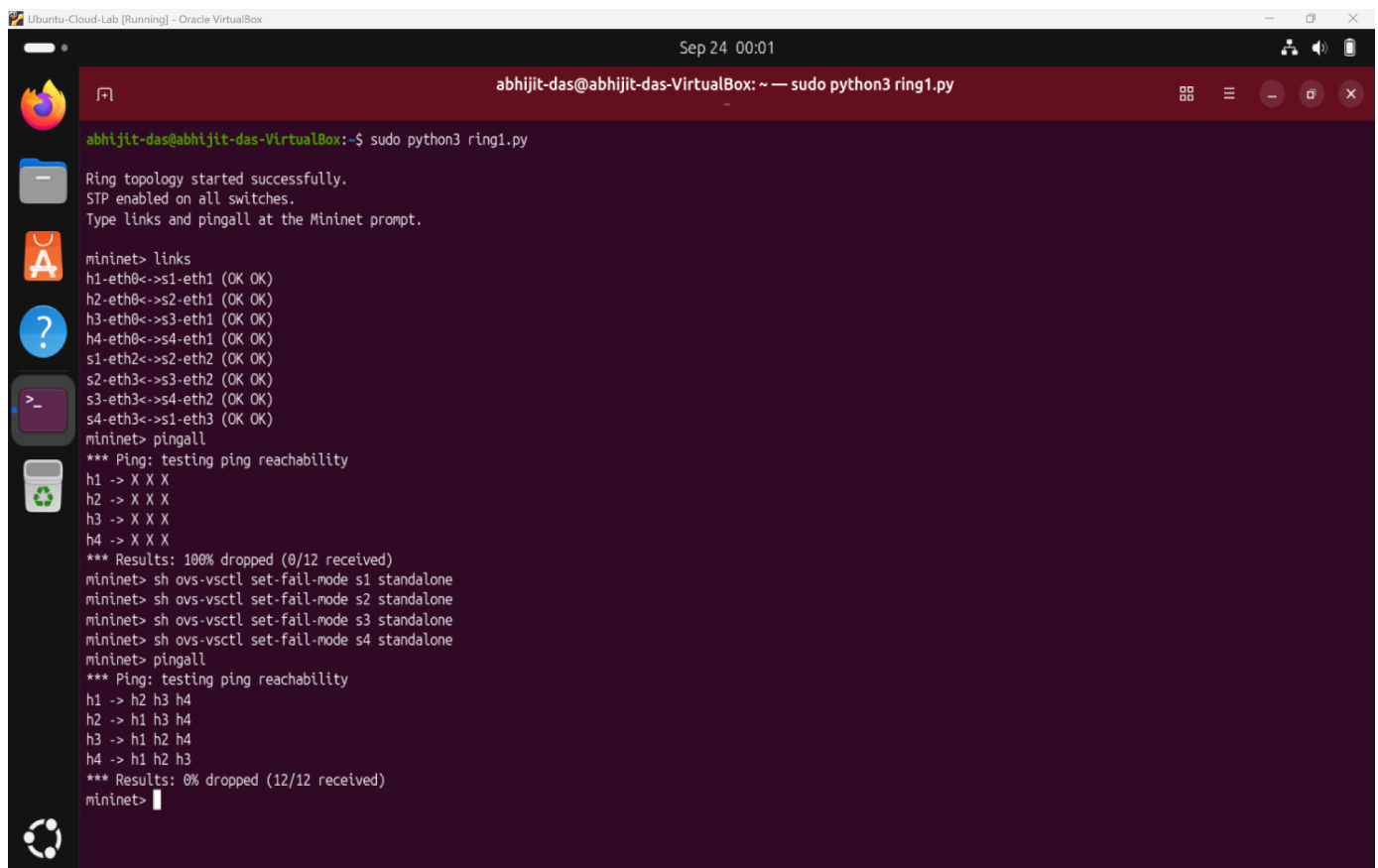
Figure 7: Configuration, links and connectivity verification of Tree topology with 8 hosts



```
abhiжит-das@abhiжит-das-VirtualBox: ~ — sudo mn -topo tree,depth=3,fanout=2
abhiжит-das@abhiжит-das-VirtualBox:~$ sudo mn --topo tree,depth=3,fanout=2
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5 h6 h7 h8
*** Adding switches:
s1 s2 s3 s4 s5 s6 s7
*** Adding links:
(s1, s2) (s1, s5) (s2, s3) (s2, s4) (s3, h1) (s3, h2) (s4, h3) (s4, h4) (s5, s6) (s5, s7) (s6, h5) (s6, h6) (s7, h7) (s7, h8)
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8
*** Starting controller

*** Starting 7 switches
s1 s2 s3 s4 s5 s6 s7 ...
*** Starting CLI:
mininet> links
s1-eth1<->s2-eth3 (OK OK)
s1-eth2<->s5-eth3 (OK OK)
s2-eth1<->s3-eth3 (OK OK)
s2-eth2<->s4-eth3 (OK OK)
s3-eth1<->h1-eth0 (OK OK)
s3-eth2<->h2-eth0 (OK OK)
s4-eth1<->h3-eth0 (OK OK)
s4-eth2<->h4-eth0 (OK OK)
s5-eth1<->s6-eth3 (OK OK)
s5-eth2<->s7-eth3 (OK OK)
s6-eth1<->h5-eth0 (OK OK)
s6-eth2<->h6-eth0 (OK OK)
s7-eth1<->h7-eth0 (OK OK)
s7-eth2<->h8-eth0 (OK OK)
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8
h2 -> h1 h3 h4 h5 h6 h7 h8
h3 -> h1 h2 h4 h5 h6 h7 h8
h4 -> h1 h2 h3 h5 h6 h7 h8
h5 -> h1 h2 h3 h4 h6 h7 h8
h6 -> h1 h2 h3 h4 h5 h7 h8
h7 -> h1 h2 h3 h4 h5 h6 h8
h8 -> h1 h2 h3 h4 h5 h6 h7
*** Results: 0% dropped (56/56 received)
mininet>
```

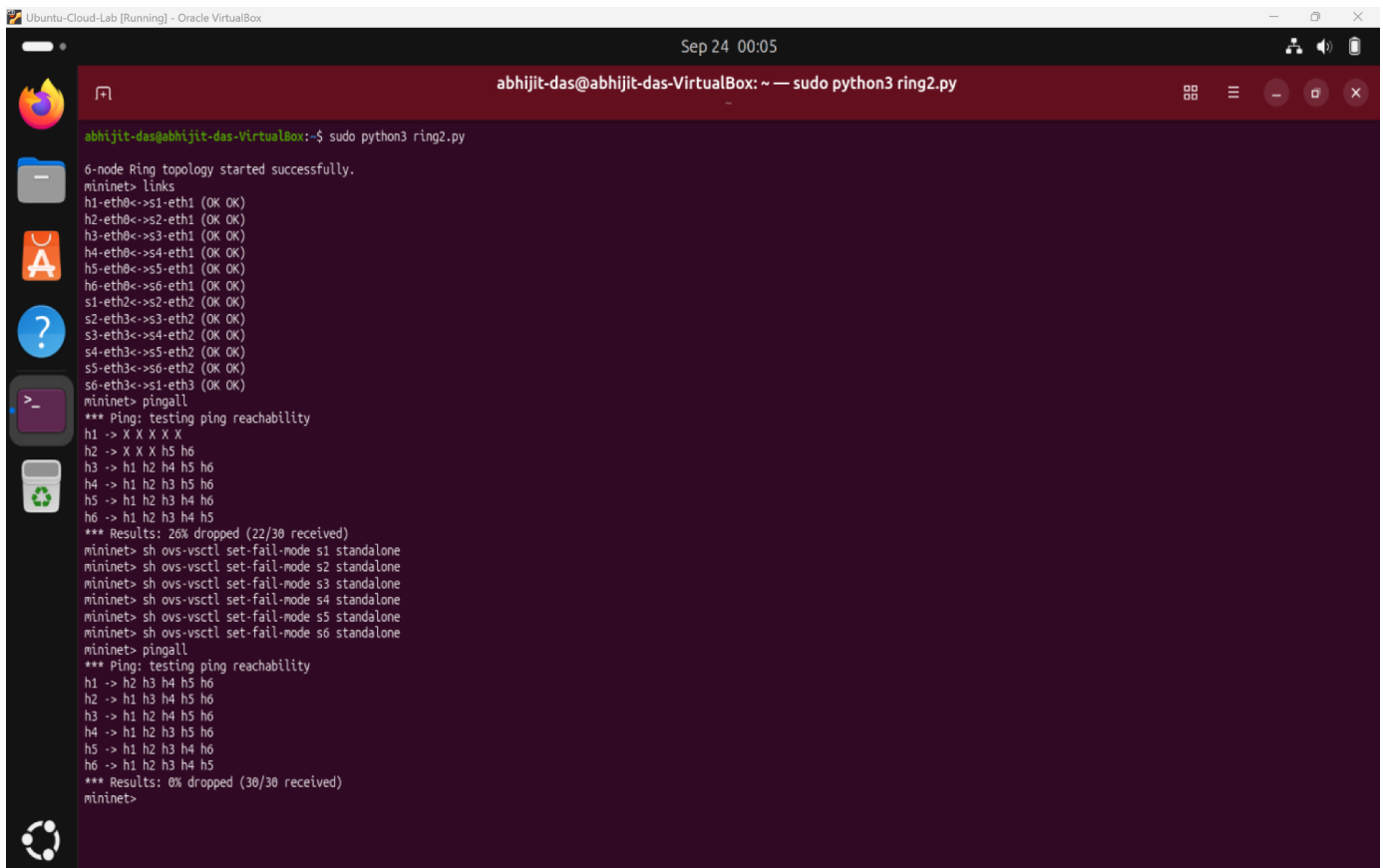
Figure 8: Execution, links and connectivity verification of Ring topology with 4 nodes



```
abhihit-das@abhihit-das-VirtualBox: ~ -- sudo python3 ring1.py
abhihit-das@abhihit-das-VirtualBox:~$ sudo python3 ring1.py
Ring topology started successfully.
STP enabled on all switches.
Type links and pingall at the Mininet prompt.

mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s2-eth1 (OK OK)
h3-eth0<->s3-eth1 (OK OK)
h4-eth0<->s4-eth1 (OK OK)
s1-eth2<->s2-eth2 (OK OK)
s2-eth3<->s3-eth2 (OK OK)
s3-eth3<->s4-eth2 (OK OK)
s4-eth3<->s1-eth3 (OK OK)
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X X
h2 -> X X X
h3 -> X X X
h4 -> X X X
*** Results: 100% dropped (0/12 received)
mininet> sh ovs-vsctl set-fail-mode s1 standalone
mininet> sh ovs-vsctl set-fail-mode s2 standalone
mininet> sh ovs-vsctl set-fail-mode s3 standalone
mininet> sh ovs-vsctl set-fail-mode s4 standalone
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet>
```

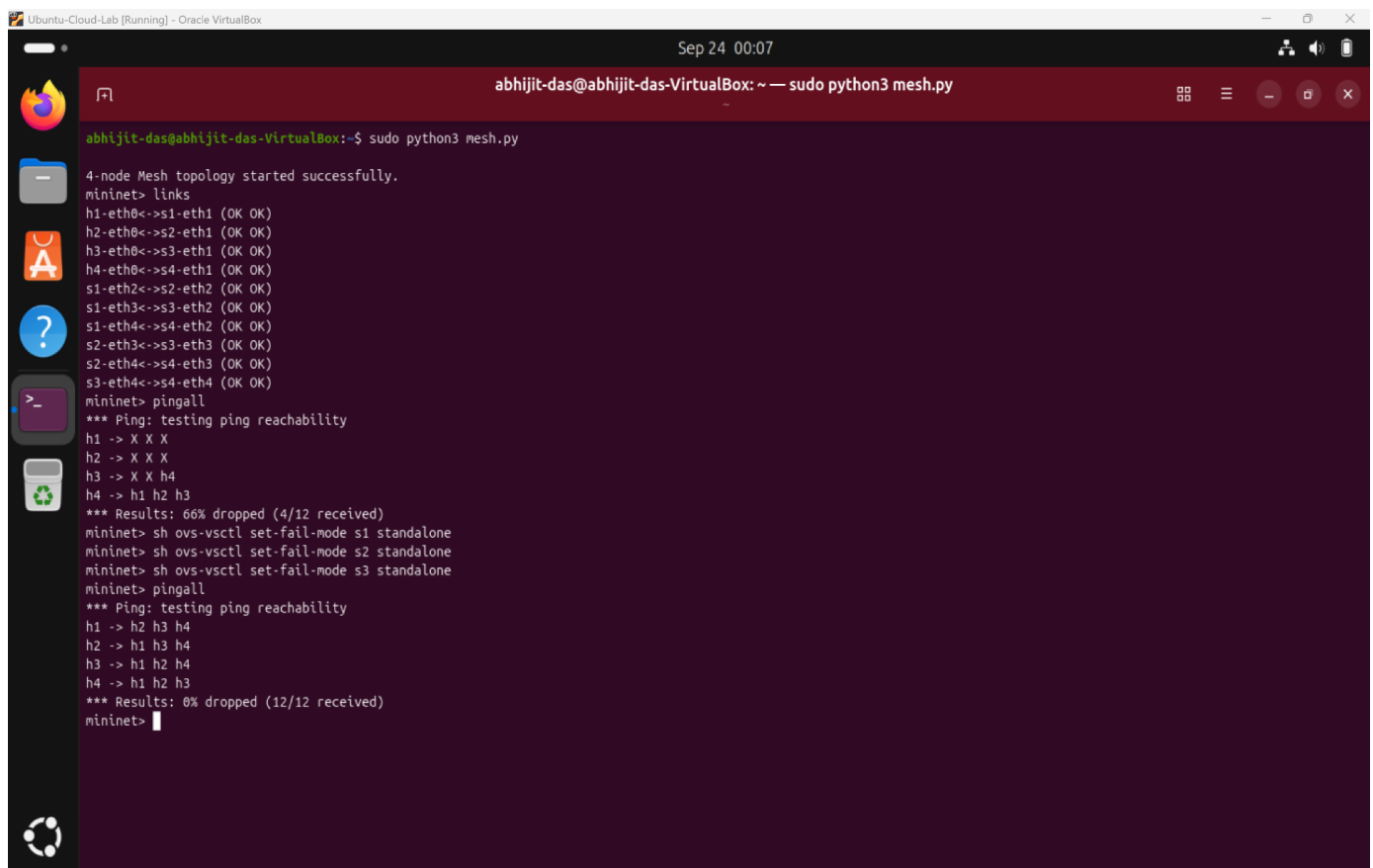
Figure 9: Execution, links and successful connectivity verification of Ring topology with 6 nodes



The screenshot shows a terminal window titled "abhiжит-das@abhiжит-das-VirtualBox: ~ — sudo python3 ring2.py". The terminal output displays the successful execution of a 6-node Ring topology. It shows the configuration of links between nodes h1 through h6 and s1 through s6, followed by a pingall test. The results show that 26% of the ping attempts were dropped (22/30 received). The user then sets the fail mode for each node to standalone and runs another pingall test, which shows 0% dropped (30/30 received).

```
abhiжит-das@abhiжит-das-VirtualBox:~$ sudo python3 ring2.py
6-node Ring topology started successfully.
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s2-eth1 (OK OK)
h3-eth0<->s3-eth1 (OK OK)
h4-eth0<->s4-eth1 (OK OK)
h5-eth0<->s5-eth1 (OK OK)
h6-eth0<->s6-eth1 (OK OK)
s1-eth2<->s2-eth2 (OK OK)
s2-eth3<->s3-eth2 (OK OK)
s3-eth3<->s4-eth2 (OK OK)
s4-eth3<->s5-eth2 (OK OK)
s5-eth3<->s6-eth2 (OK OK)
s6-eth3<->s1-eth3 (OK OK)
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X X X X
h2 -> X X X h5 h6
h3 -> h1 h2 h4 h5 h6
h4 -> h1 h2 h3 h5 h6
h5 -> h1 h2 h3 h4 h6
h6 -> h1 h2 h3 h4 h5
*** Results: 26% dropped (22/30 received)
mininet> sh ovs-vsctl set-fail-mode s1 standalone
mininet> sh ovs-vsctl set-fail-mode s2 standalone
mininet> sh ovs-vsctl set-fail-mode s3 standalone
mininet> sh ovs-vsctl set-fail-mode s4 standalone
mininet> sh ovs-vsctl set-fail-mode s5 standalone
mininet> sh ovs-vsctl set-fail-mode s6 standalone
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6
h2 -> h1 h3 h4 h5 h6
h3 -> h1 h2 h4 h5 h6
h4 -> h1 h2 h3 h5 h6
h5 -> h1 h2 h3 h4 h6
h6 -> h1 h2 h3 h4 h5
*** Results: 0% dropped (30/30 received)
mininet>
```

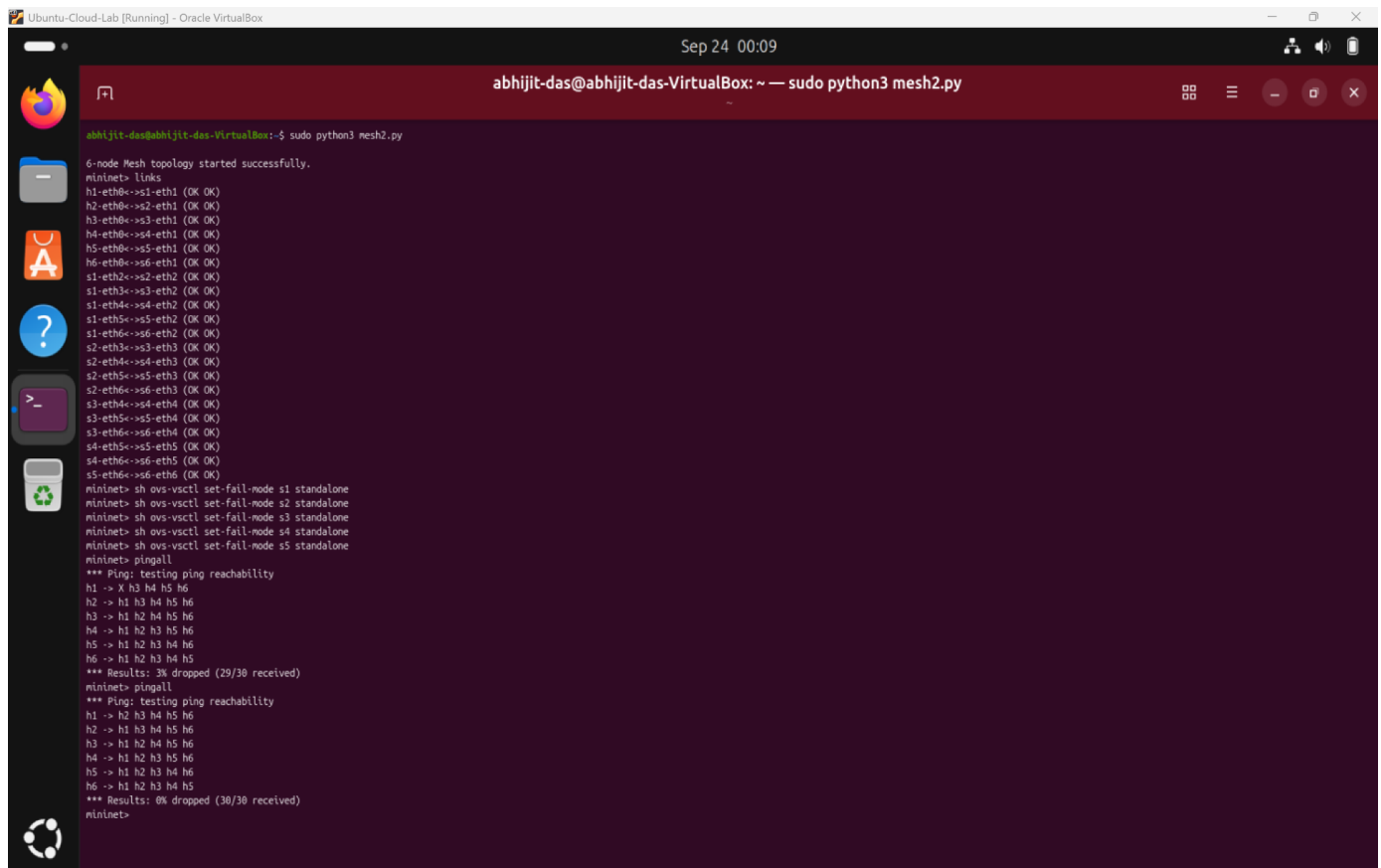
Figure 10: Execution, links and connectivity verification of Mesh topology with 4 nodes



The screenshot shows a terminal window titled "abhihit-das@abhihit-das-VirtualBox: ~ — sudo python3 mesh.py". The terminal output is as follows:

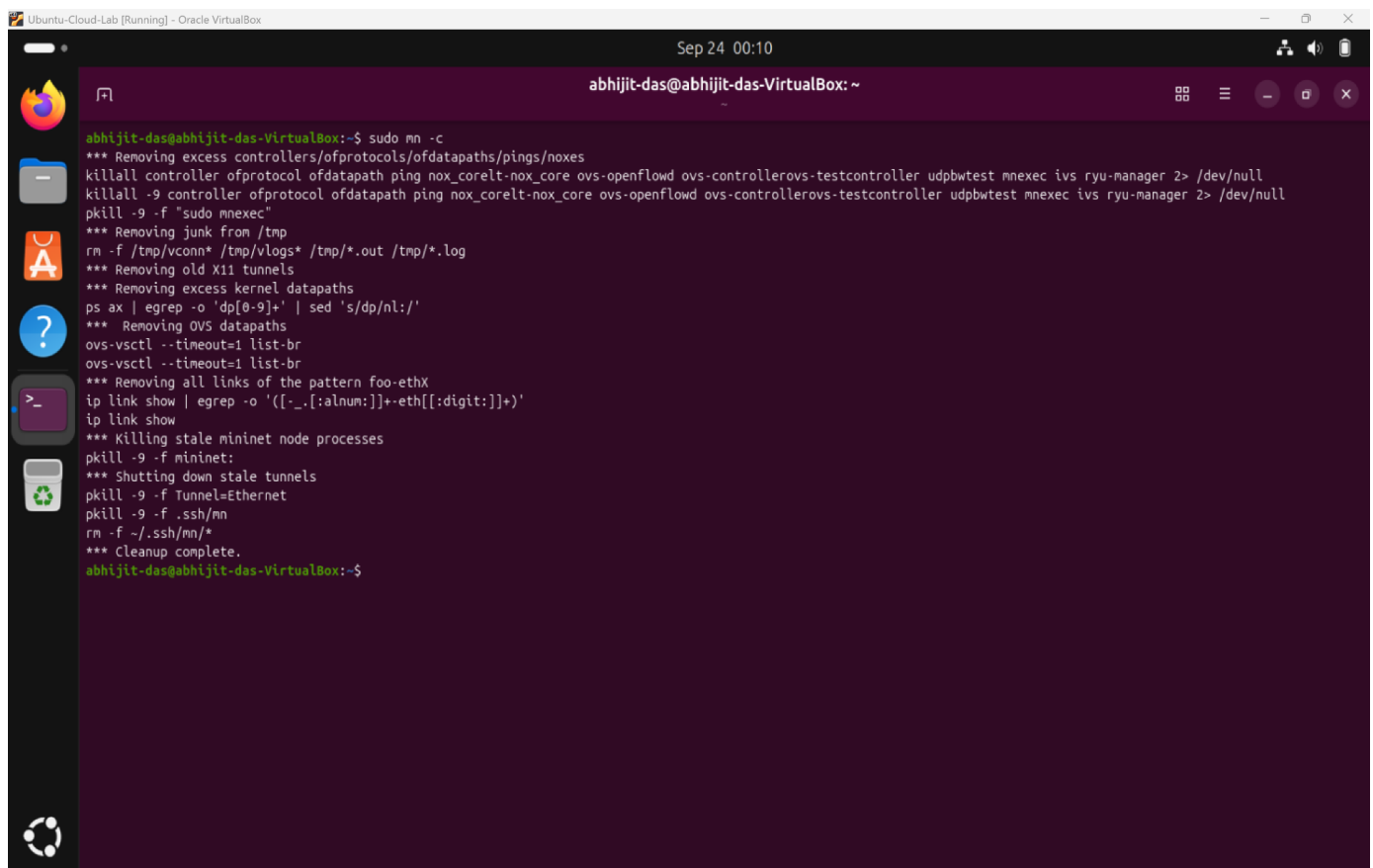
```
abhihit-das@abhihit-das-VirtualBox:~$ sudo python3 mesh.py
4-node Mesh topology started successfully.
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s2-eth1 (OK OK)
h3-eth0<->s3-eth1 (OK OK)
h4-eth0<->s4-eth1 (OK OK)
s1-eth2<->s2-eth2 (OK OK)
s1-eth3<->s3-eth2 (OK OK)
s1-eth4<->s4-eth2 (OK OK)
s2-eth3<->s3-eth3 (OK OK)
s2-eth4<->s4-eth3 (OK OK)
s3-eth4<->s4-eth4 (OK OK)
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X X
h2 -> X X X
h3 -> X X h4
h4 -> h1 h2 h3
*** Results: 66% dropped (4/12 received)
mininet> sh ovs-vsctl set-fail-mode s1 standalone
mininet> sh ovs-vsctl set-fail-mode s2 standalone
mininet> sh ovs-vsctl set-fail-mode s3 standalone
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet>
```

Figure 11: Execution, links and connectivity verification of Mesh topology with 6 nodes



```
abhiжит-das@abhiжит-das-VirtualBox: ~ — sudo python3 mesh2.py
6-node Mesh topology started successfully.
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s2-eth1 (OK OK)
h3-eth0<->s3-eth1 (OK OK)
h4-eth0<->s4-eth1 (OK OK)
h5-eth0<->s5-eth1 (OK OK)
h6-eth0<->s6-eth1 (OK OK)
s1-eth2<->s2-eth2 (OK OK)
s1-eth3<->s3-eth2 (OK OK)
s1-eth4<->s4-eth2 (OK OK)
s1-eth5<->s5-eth2 (OK OK)
s1-eth6<->s6-eth2 (OK OK)
s2-eth3<->s3-eth3 (OK OK)
s2-eth4<->s4-eth3 (OK OK)
s2-eth5<->s5-eth3 (OK OK)
s2-eth6<->s6-eth3 (OK OK)
s3-eth4<->s4-eth4 (OK OK)
s3-eth5<->s5-eth4 (OK OK)
s3-eth6<->s6-eth4 (OK OK)
s4-eth5<->s5-eth5 (OK OK)
s4-eth6<->s6-eth5 (OK OK)
s5-eth6<->s6-eth6 (OK OK)
mininet> sh ovs-vsctl set-fail-mode s1 standalone
mininet> sh ovs-vsctl set-fail-mode s2 standalone
mininet> sh ovs-vsctl set-fail-mode s3 standalone
mininet> sh ovs-vsctl set-fail-mode s4 standalone
mininet> sh ovs-vsctl set-fail-mode s5 standalone
mininet> pingall
*** Ping: testing ping reachability
h1 -> X h3 h4 h5 h6
h2 -> h1 h3 h4 h5 h6
h3 -> h1 h2 h4 h5 h6
h4 -> h1 h2 h3 h5 h6
h5 -> h1 h2 h3 h4 h6
h6 -> h1 h2 h3 h4 h5
*** Results: 3% dropped (29/30 received)
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6
h2 -> h1 h3 h4 h5 h6
h3 -> h1 h2 h4 h5 h6
h4 -> h1 h2 h3 h5 h6
h5 -> h1 h2 h3 h4 h6
h6 -> h1 h2 h3 h4 h5
*** Results: 0% dropped (30/30 received)
mininet>
```

Figure 12: Cleaning and stopping the Mininet network after completion of the experiment



```
abhihit-das@abhihit-das-VirtualBox:~$ sudo mn -c
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflowd ovs-controller ovs-testcontroller udptest mnexec ixs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflowd ovs-controller ovs-testcontroller udptest mnexec ixs ryu-manager 2> /dev/null
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([a-z0-9:]+)-eth[0-9]+'
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f Tunnel=Ethernet
pkill -9 -f .ssh/mn
rm -f ~/.ssh/mn/*
*** Cleanup complete.
abhihit-das@abhihit-das-VirtualBox:~$
```